

Day 37 – Docker Revision & Cheat Sheet

Quick-reference commands, self-assessment, and review questions for Days 29–36.

Container Commands

Command	Purpose
<code>docker run -it ubuntu bash</code>	Run a container interactively with a shell.
<code>docker run -d nginx</code>	Run a container in detached/background mode.
<code>docker ps</code>	List running containers.
<code>docker ps -a</code>	List all containers, including stopped ones.
<code>docker stop</code>	Gracefully stop a running container.
<code>docker rm</code>	Remove a stopped container.
<code>docker exec -it bash</code>	Open an interactive shell inside a running container.
<code>docker logs</code>	View container stdout/stderr logs.

Image Commands

Command	Purpose
<code>docker build -t : .</code>	Build an image from a Dockerfile and assign a tag.
<code>docker pull</code>	Download an image from a registry such as Docker Hub.
<code>docker push /:</code>	Push a tagged image to a registry.
<code>docker tag :</code>	Create another tag/reference for an image.
<code>docker images</code>	List local Docker images.
<code>docker rmi</code>	Remove a local image.

Volume Commands

Command	Purpose
<code>docker volume create</code>	Create a named volume managed by Docker.
<code>docker volume ls</code>	List Docker volumes.
<code>docker volume inspect</code>	Show detailed volume configuration and mount information.
<code>docker volume rm</code>	Remove a volume.

Network Commands

Command	Purpose
<code>docker network create</code>	Create a custom Docker network.
<code>docker network ls</code>	List Docker networks.
<code>docker network inspect</code>	Show network configuration and connected containers.
<code>docker network connect</code>	Connect an existing container to a network.

Compose Commands

Command	Purpose
<code>docker compose up -d</code>	Create and start Compose services in detached mode.
<code>docker compose down</code>	Stop and remove Compose services and networks.
<code>docker compose ps</code>	Show the status of Compose services.

Command	Purpose
<code>docker compose logs -f</code>	Follow logs from Compose services.
<code>docker compose build</code>	Build images defined by the Compose file.

Cleanup Commands

Command	Purpose
<code>docker system df</code>	Show Docker disk usage by images, containers, volumes, and build cache.
<code>docker system prune</code>	Remove unused Docker resources; review the prompt before confirming.
<code>docker system prune -a</code>	Remove unused resources including unused images; use with care.

Dockerfile Instructions

Command	Purpose
FROM	Set the base image.
RUN	Execute a command during image build.
COPY	Copy files/directories from the build context into the image.
WORKDIR	Set the working directory for subsequent instructions and the default runtime directory.
EXPOSE	Document the port the application listens on; it does not publish the port.
CMD ["command", "arg"]	Set the default command/arguments; can be overridden at runtime.
ENTRYPOINT ["command"]	Define the main executable; runtime arguments are normally appended to it.

Self-Assessment Checklist

Status	Skill
■ Can do / ■ Shaky / ■ Haven't done	Running containers: interactive and detached
■ Can do / ■ Shaky / ■ Haven't done	Managing containers and images: list, stop, remove
■ Can do / ■ Shaky / ■ Haven't done	Explaining image layers and build caching
■ Can do / ■ Shaky / ■ Haven't done	Writing Dockerfiles with key instructions
■ Can do / ■ Shaky / ■ Haven't done	Understanding CMD vs ENTRYPOINT
■ Can do / ■ Shaky / ■ Haven't done	Building and tagging custom images
■ Can do / ■ Shaky / ■ Haven't done	Creating and using named volumes and bind mounts
■ Can do / ■ Shaky / ■ Haven't done	Creating custom networks and connecting containers
■ Can do / ■ Shaky / ■ Haven't done	Writing docker-compose.yml for multi-container applications
■ Can do / ■ Shaky / ■ Haven't done	Using environment variables and .env files in Compose
■ Can do / ■ Shaky / ■ Haven't done	Writing and understanding multi-stage Dockerfiles
■ Can do / ■ Shaky / ■ Haven't done	Pushing images to Docker Hub
■ Can do / ■ Shaky / ■ Haven't done	Using healthchecks and depends_on

Quick-Fire Questions & Answers

1. What is the difference between an image and a container?

An image is a read-only template containing application code and dependencies. A container is a created/running instance of an image.

2. What happens to data when a container is removed?

Data in the container's writable layer is removed with the container. Data in a named volume or bind mount persists separately.

3. How do containers communicate on the same custom network?

They communicate over the Docker network and can resolve each other by container/service name using Docker's built-in DNS.

4. What is the difference between `docker compose down -v` and `docker compose down`?

`down` removes Compose containers and networks. `down -v` also removes the project's associated volumes.

5. Why are multi-stage builds useful?

They separate build and runtime environments so the final image can contain only the runtime and required application artifacts, reducing unnecessary image size and dependencies.

6. What is the difference between `COPY` and `ADD`?

`COPY` performs straightforward file/directory copying. `ADD` has additional behavior such as handling local tar archives; `COPY` is generally preferred for normal copies.

7. What does `-p 8080:80` mean?

It maps host port 8080 to container port 80. The application inside the container must actually listen on port 80.

8. How do you check Docker disk space usage?

Use `docker system df`.

Key Revision Notes

- **Image layers:** Docker images are composed of read-only layers. Unchanged build layers can be reused from cache.
- **CMD vs ENTRYPOINT:** `CMD` supplies defaults that can be overridden. `ENTRYPOINT` defines the main executable; `CMD` can provide its default arguments.
- **Volumes vs bind mounts:** Named volumes are managed by Docker; bind mounts map a host filesystem path directly into a container.
- **Port mapping:** `-p 8080:80` means host port 8080 → container port 80. `EXPOSE` alone does not publish a port.
- **Compose:** Services on the same Compose network can reach one another by service name.
- **Healthchecks:** A healthcheck tells Docker how to determine whether a container is healthy. `depends_on` defines service dependency and can be combined with health conditions.